SOAC SPECIFICATION

CLAW Playbook
Schema v1.0

Continuous, Logic-Adaptive Workflow Engine

Publisher: SOaC Core Team

Version: 1.0.0

Date: March 2026

License: Apache 2.0

Repository: github.com/ge0mant1s/SOaC-Enterprise

Security Operations as Code · Distributed Intelligence Architecture

Table of Contents

1 Introduction 3

2 Schema Definition 4

3 Field Reference 6

4 Execution Lifecycle 7

5 Governance & Blast Radius 8

6 Example: Revoke Compromised Sessions 9

7 Example: Isolate Compromised Host 11

8 Detection Schema Reference 13

9 Versioning & Changelog 14

1. Introduction

The **CLAW Engine** (Continuous, Logic-Adaptive Workflow) is the automated response layer of the SOaC Distributed Intelligence Architecture. It sits at the intersection of The Body (detection telemetry), The Brain (AI-driven reasoning), and The Nervous System (edge enforcement).

CLAW playbooks are **YAML contracts** that define exactly how the system responds to a threat — what steps to execute, what approvals are required, how failures are handled, and when to roll back. Every playbook is version-controlled, peer-reviewed, and immutably audited.

> **Design Principle:** A CLAW playbook should be readable by a SOC analyst, reviewable by an engineer, and auditable by a compliance team — all from the same YAML file.

Scope

This specification covers:

- The complete CLAW Playbook YAML schema (Section 2)
- Field-by-field reference with validation rules (Section 3)
- Execution lifecycle and state machine (Section 4)
- Governance controls and blast radius governance (Section 5)
- Two complete playbook examples with commentary (Sections 6–7)
- Detection schema reference for Body-to-CLAW integration (Section 8)
- Versioning policy and changelog (Section 9)

Audience

Role	How to use this spec
SOC Analyst / IR	Understand playbook behavior, failure modes, and rollback triggers
Detection Engineer	Write detection rules that trigger CLAW playbooks correctly
Platform Engineer	Implement CLAW execution engine, integrate with APIs
CISO / Compliance	Review governance controls, blast radius, and audit guarantees

2. Schema Definition

Every CLAW playbook must conform to the following YAML structure. Fields marked `required` must be present for the playbook to validate.

```yaml
# CLAW Playbook Schema v1.0
apiVersion: claw.soac.io/v1          # required
kind: Playbook                     # required

metadata:
  name: string                     # required · unique identifier
  version: string                  # required · semver (e.g., "1.2.0")
  author: string                   # required · author ID
  created: datetime                # ISO 8601 timestamp
  tags: string[]                   # classification tags
  mitre_attack: string[]            # required · MITRE ATT&CK IDs
  severity: CRITICAL | HIGH | MEDIUM | LOW

spec:
  description: string               # human-readable description

  trigger:
    source: BODY | BRAIN | MANUAL | SCHEDULED
    conditions:
      - field: string
        operator: eq | neq | gt | lt | contains | regex
        value: any
    cooldown: duration              # e.g., "5m", "1h"

  inputs:
    - name: string
      type: string | number | boolean | array | object
      required: boolean
      source: detection | enrichment | manual | config
      default: any

  steps:
    - id: string                   # required · unique step ID
      name: string                 # human-readable name
      action: api_call | script | decision | notify
      target: string               # target system
      params: object               # action-specific parameters
      timeout: duration             # max execution time
      retry:
        max_attempts: number
        backoff: linear | exponential
      on_failure: continue | abort | rollback | escalate
      conditions:
        - when: string             # condition expression
          then: string             # step to execute

  outputs:
    - name: string
      type: string | number | boolean | array | object
      source: string               # step ID that produces output

  rollback:
    enabled: boolean
    steps: [...]                      # same structure as spec.steps
```

```
  governance:
    requires_approval: boolean
    approval_timeout: duration
    max_blast_radius: single_user | team | org | global
    audit_level: minimal | standard | verbose
    brain_oversight: boolean       # AI must validate before exec
```

3. Field Reference

3.1 Metadata

Field	Type	Required	Description
`apiVersion`	string	Yes	Must be `claw.soac.io/v1`
`kind`	string	Yes	Must be `Playbook`
`metadata.name`	string	Yes	Unique kebab-case identifier, e.g. `revoke-compromised-sessions`
`metadata.version`	string	Yes	Semantic versioning (MAJOR.MINOR.PATCH)
`metadata.mitre_attack`	string[]	Yes	MITRE ATT&CK technique IDs (e.g., T1557.001)
`metadata.severity`	enum	Yes	CRITICAL, HIGH, MEDIUM, or LOW

3.2 Trigger

Field	Type	Description
`trigger.source`	enum	Which SOaC component initiates: BODY, BRAIN, MANUAL, SCHEDULED
`trigger.conditions`	array	All conditions must match (AND logic). Supports operators: eq, neq, gt, lt, contains, regex
`trigger.cooldown`	duration	Minimum time between executions. Prevents rapid re-triggering (e.g., "5m", "1h")

3.3 Steps

Field	Type	Description
`steps[].action`	enum	**api_call**: HTTP call to external service. **script**: Execute command on target. **decision**: Branch logic. **notify**: Send alert.
`steps[].on_failure`	enum	**continue**: skip and proceed. **abort**: halt playbook. **rollback**: execute rollback steps. **escalate**: alert human operator.
`steps[].retry`	object	`max_attempts` (1–10) and `backoff` (linear or exponential delay between retries)
`steps[].timeout`	duration	Per-step timeout. Execution halts if exceeded. Example: "30s", "2m"

3.4 Governance

Field	Type	Description

`brain_oversight`	boolean	When `true`, The Brain (AI) must validate and approve before execution begins
`max_blast_radius`	enum	Limits scope: `single_user` (safest), `team`, `org`, `global` (most impact)
`requires_approval`	boolean	If `true`, human approval required before execution (subject to `approval_timeout`)
`audit_level`	enum	`verbose`: full I/O logging. `standard`: summary. `minimal`: status only.

4. Execution Lifecycle

Every CLAW playbook execution follows a deterministic 6-phase lifecycle:

Phase 1 — TRIGGER

A detection event from The Body (or manual/scheduled source) matches the playbook's `trigger.conditions`. The engine checks the `cooldown` window — if a previous execution occurred within the cooldown period, the trigger is suppressed and logged.

Phase 2 — VALIDATE

If `governance.brain_oversight` is `true`, the playbook context (detection data, proposed actions, blast radius) is sent to The Brain for AI-assisted validation. The Brain evaluates risk, checks for prompt injection, and returns APPROVE or DENY with an audit hash.

> **Important:** If `requires_approval` is `true`, execution blocks until a human operator approves or the `approval_timeout` expires. Timeout behavior is configurable: default is to abort.

Phase 3 — EXECUTE

Steps execute sequentially. Each step's `conditions` are evaluated before execution. Variable interpolation (`${var_name{'}'}`) resolves inputs and previous step outputs. Retry logic applies per step configuration.

Phase 4 — ENFORCE

Steps targeting `edge_enforcement` trigger The Nervous System. Enforcement actions are HMAC-signed and propagated to all edge nodes. Each node confirms enforcement and returns an enforcement ID.

Phase 5 — REPORT

All outputs are collected, an immutable audit record is generated (SHA-256 hash), and notifications are dispatched. The execution summary is stored in the audit log with full I/O traces if `audit_level: verbose`.

Phase 6 — ROLLBACK (conditional)

If any step with `on_failure: rollback` fails, the engine executes `rollback.steps` in reverse order. Rollback steps follow the same retry and timeout logic as regular steps. Rollback execution is itself audited.

State Machine

```
PENDING → VALIDATING → EXECUTING → ENFORCING → REPORTING → COMPLETE
                ↓              ↓           ↓
              DENIED       ROLLING_BACK  ESCALATED
```

```
↓
ROLLED_BACK
```

5. Governance & Blast Radius

CLAW enforces governance at multiple levels to prevent runaway automation and ensure auditability.

5.1 Blast Radius Matrix

Level	Scope	Example Actions	Brain Oversight
`single_user`	One user account	Revoke sessions, reset MFA	Optional
`team`	Group/OU	Quarantine shared mailbox, disable group	Recommended
`org`	Entire organization	Block IP range, disable OAuth app	Required
`global`	Cross-tenant / infra	DNS sinkhole, global firewall rule	Required + Human approval

5.2 AI Governance (The Brain)

When `brain_oversight: true`, The Brain performs the following checks before approving execution:

- **Prompt injection scan:** Input sanitization, semantic analysis, canary token detection
- **Risk assessment:** Correlates detection confidence with blast radius to compute execution risk
- **Behavioral baseline:** Compares proposed response to historical patterns
- **PII detection:** Scans outputs for personally identifiable information leakage

All Brain decisions are logged with an immutable audit hash (SHA-256) for compliance.

5.3 Failure Handling Strategy

on_failure	Behavior	Use When
`continue`	Log failure, skip step, proceed to next	Non-critical enrichment or notification steps
`abort`	Halt playbook immediately, no rollback	Verification steps where failure means invalid context
`rollback`	Execute rollback.steps in reverse order	Containment steps where partial execution is dangerous
`escalate`	Halt playbook, alert human operator via SOC channel	Critical enforcement steps requiring manual intervention

6. Example: Revoke Compromised Sessions

CRITICAL T1557.001 T1078.004 identity aitm

This playbook responds to AitM phishing or session token theft by revoking all active sessions and enforcing re-authentication across identity providers.

```
apiVersion: claw.soac.io/v1
kind: Playbook
metadata:
  name: revoke-compromised-sessions
  version: "1.2.0"
  author: soac-core-team
  created: "2026-01-15T00:00:00Z"
  tags: [session-hijack, identity, aitm]
  mitre_attack: [T1557.001, T1078.004]
  severity: CRITICAL

spec:
  description: Revokes all active sessions for a compromised user detected via AitM phishi
ng or session token theft.

  trigger:
    source: BODY
    conditions:
      - field: detection_type
        operator: eq
        value: "aitm_session_hijack"
      - field: risk_score
        operator: eq
        value: "CRITICAL"
    cooldown: 5m

  inputs:
    - name: actor_email     | type: string  | required: true  | source: detection
    - name: session_ids     | type: array   | required: true  | source: detection
    - name: source_ips      | type: array   | required: false | source: enrichment

  steps:
    - id: enrich_user
      name: Enrich User Context
      action: api_call     | target: okta
      params: { endpoint: /api/v1/users/${actor_email{'}'}, method: GET }
      timeout: 10s | retry: { max_attempts: 3, backoff: exponential }
      on_failure: abort

    - id: revoke_okta_sessions
      name: Revoke All Okta Sessions
      action: api_call     | target: okta
      params: { endpoint: /api/v1/users/${user_id{'}'}/sessions, method: DELETE }
      timeout: 15s | retry: { max_attempts: 2, backoff: linear }
      on_failure: escalate

    - id: revoke_azure_tokens
      name: Revoke Azure AD Refresh Tokens
      action: api_call     | target: microsoft_graph
      params: { endpoint: /v1.0/users/${actor_email{'}'}/revokeSignInSessions, method: POS
T }
      on_failure: continue
```

```
    - id: notify_edge
      name: Push Block to Edge Enforcement
      action: api_call     | target: edge_enforcement
      params: { action: block_session, scope: user, duration: 24h, hmac_signed: true }
      on_failure: escalate

    - id: notify_soc
      name: Alert SOC Team
      action: notify       | target: slack
      params: { channel: #soc-critical }
      on_failure: continue

  rollback:
    enabled: true
    steps:
      - id: restore_sessions | action: api_call | target: okta
        params: { endpoint: /api/v1/users/${user_id{'}'}/lifecycle/activate, method: POST
}

  governance:
    requires_approval: false
    max_blast_radius: single_user
    audit_level: verbose
    brain_oversight: true
```

Key decisions: `on_failure: abort` on the first enrichment step ensures we don't revoke sessions if we can't confirm user identity. `brain_oversight: true` forces AI validation for every execution. Blast radius is limited to `single_user`.

7. Example: Isolate Compromised Host

CRITICAL T1486 T1059 T1068 endpoint ransomware

This playbook isolates a compromised endpoint detected by CrowdStrike, applies network containment, preserves forensic artifacts, and creates an IR ticket.

```
apiVersion: claw.soac.io/v1
kind: Playbook
metadata:
  name: isolate-compromised-host
  version: "1.1.0"
  author: soac-core-team
  mitre_attack: [T1486, T1059, T1068]
  severity: CRITICAL

spec:
  description: Isolates a compromised endpoint. Applies network containment, kills malicio
us processes, and preserves forensic artifacts.

  trigger:
    source: BODY
    conditions:
      - field: detection_type  | operator: contains  | value: "ransomware"
      - field: confidence      | operator: gt        | value: 85
    cooldown: 2m

  inputs:
    - name: host_id        | type: string  | required: true
    - name: hostname       | type: string  | required: true
    - name: malicious_pids | type: array   | required: false
    - name: threat_actor   | type: string  | required: false

  steps:
    - id: verify_host       # Verify host identity before containment
      action: api_call | target: crowdstrike
      params: { endpoint: /devices/entities/devices/v2, method: GET }
      on_failure: abort

    - id: contain_host      # Network isolation via CrowdStrike
      action: api_call | target: crowdstrike
      params: { action_name: contain, ids: [${host_id{'}'}] }
      timeout: 30s | retry: { max_attempts: 3, backoff: exponential }
      on_failure: escalate

    - id: kill_processes    # Terminate malicious PIDs (conditional)
      action: script   | target: crowdstrike_rtr
      params: { command: kill, pids: ${malicious_pids{'}'} }
      conditions: [{ when: "malicious_pids ≠ null", then: execute }]
      on_failure: continue

    - id: snapshot_memory   # Forensic memory capture
      action: script   | target: crowdstrike_rtr
      params: { command: memdump }
      timeout: 120s | on_failure: continue

    - id: push_edge_block   # Edge firewall enforcement
      action: api_call | target: edge_enforcement
      params: { action: block_host, scope: network, duration: 48h, hmac_signed: true }
      on_failure: escalate
```

```
    - id: create_ticket     # JIRA incident ticket
      action: api_call | target: jira
      params: { project: SOC, issue_type: Incident, priority: Critical }
      on_failure: continue

  rollback:
    enabled: true
    steps:
      - id: lift_containment  | action: api_call | target: crowdstrike
        params: { action_name: lift_containment, ids: [${host_id{'}'}] }
      - id: remove_edge_block | action: api_call | target: edge_enforcement
        params: { action: unblock_host, identifier: ${host_id{'}'} }

  governance:
    requires_approval: false
    max_blast_radius: single_user
    audit_level: verbose
    brain_oversight: true
```

Key decisions: 6-step playbook with conditional process kill (only if PIDs known). Memory snapshot uses `on_failure: continue` — forensics shouldn't block containment. Two-step rollback lifts both CrowdStrike containment and edge firewall block.

8. Detection Schema Reference

Detection rules in The Body emit events that trigger CLAW playbooks. Every detection event must include the following fields for CLAW integration:

8.1 Required Detection Output Fields

Field	Type	Description
`detection_id`	string	Unique identifier, format: `BODY-{TYPE}-{PLATFORM}-{hash}`
`detection_type`	string	Classification key matching CLAW trigger conditions
`risk_score`	enum	CRITICAL, HIGH, MEDIUM, LOW — determines playbook selection
`confidence`	number	0–100 percentage of detection confidence
`mitre_attack`	string[]	Mapped MITRE ATT&CK technique IDs
`actor_email`	string	Affected user identifier (for identity-based playbooks)
`host_id`	string	Affected host identifier (for endpoint-based playbooks)
`timestamp`	datetime	ISO 8601 detection timestamp

8.2 Example: Splunk Detection Output (AitM)

```
| SOaC Detection Rule: AitM Phishing - Okta Session Hijack
| Platform: Splunk Enterprise Security
| MITRE ATT&CK: T1557.001 (AitM), T1078.004 (Cloud Accounts)

index=okta sourcetype=OktaIM2:log
| spath output=eventType path=eventType
| spath output=actor_email path=actor.alternateId
| spath output=client_ip path=client.ipAddress
| spath output=client_geo path=client.geographicalContext.country
| where eventType IN ("user.session.start", "user.authentication.sso")
| stats
    dc(client_ip) as unique_ips,
    dc(client_geo) as unique_geos,
    values(client_ip) as ip_list
    by actor_email
| where unique_ips > 2 AND unique_geos > 1
| eval risk_score = case(
    unique_geos ≥ 3, "CRITICAL",
    unique_ips ≥ 4, "HIGH",
    1=1, "MEDIUM")
| eval detection_id = "BODY-AITM-OKTA-" . md5(actor_email)
| table detection_id, actor_email, risk_score, unique_ips, unique_geos
```

Body → CLAW contract: The detection output fields (`detection_type` , `risk_score` , `actor_email`) directly map to the CLAW playbook's `trigger.conditions` and `inputs` . This ensures deterministic triggering without field translation.

9. Versioning & Changelog

9.1 Versioning Policy

CLAW playbook schema follows **Semantic Versioning 2.0.0**:

Change Type	Version Bump	Examples
Breaking schema change	MAJOR	Removing a required field, changing `apiVersion`
New optional field	MINOR	Adding `steps[].conditions`, new `on_failure` mode
Documentation fix	PATCH	Typo corrections, clarifying field descriptions

9.2 Individual Playbook Versioning

Each playbook carries its own `metadata.version` independent of the schema version. Playbook versions should be bumped when:

- **MAJOR:** Playbook logic fundamentally changes (e.g., different containment strategy)
- **MINOR:** New step added, existing step modified (e.g., adding MFA reset to session revocation)
- **PATCH:** Parameter tuning (e.g., adjusting timeout from 10s to 15s)

9.3 Changelog

Version	Date	Changes
`v1.0.0`	2026-03-12	Initial release of CLAW Playbook Schema specification. Includes full schema definition, field reference, governance controls, two complete playbook examples, and detection schema reference.

9.4 Planned for v1.1.0

- Parallel step execution (`steps[].parallel: true`)
- Step output piping (`steps[].pipe_to`)
- Conditional playbook chaining (`on_complete.chain`)
- Extended trigger sources (SIEM webhook, threat intel feed)
- Playbook composition / inheritance

> **Contributing:** This spec is open-source. Submit schema change proposals as GitHub Issues with the `schema-rfc` label. All breaking changes require community discussion before merging.

© 2026 SOaC Core Team · Apache 2.0 License · github.com/ge0mant1s/SOaC-Enterprise